

Replayable Construction and the Computational Basis of Abstraction

Flyxion

December 14, 2025

Abstract

Despite recent progress in large-scale learning systems, abstraction remains poorly characterized as a computational capability. Many systems exhibit strong compression, generalization, and pattern completion while failing to demonstrate robust abstraction under distributional shift or structural perturbation. This paper argues that the missing ingredient is not scale, data, or expressivity, but replayable construction. We formalize abstraction as the stabilization of semantic equivalence across alternative construction histories and show that such stabilization requires access to replayable causal traces. Compression alone is insufficient in principle. We present a minimal formal framework that distinguishes abstraction from representation learning, clarifies the limits of current architectures, and provides a diagnostic criterion for abstraction in artificial systems.

1 Introduction

Modern artificial intelligence systems achieve impressive performance across a wide range of tasks, often surpassing human benchmarks in perception, control, and language processing. These successes are frequently attributed to advances in representation learning, large-scale optimization, and statistical generalization. Yet despite these gains, a persistent gap remains between systems that interpolate effectively within known regimes and systems that exhibit robust abstraction.

This gap manifests in familiar ways: sensitivity to spurious correlations, brittle failure under minor perturbations, difficulty transferring knowledge across domains, and limited capacity for principled recombination. These shortcomings persist even in systems that exhibit strong compression and generalization by conventional metrics.

This paper argues that these failures are not accidental but structural. The prevailing paradigm equates abstraction with compression, dimensionality reduction, or representational efficiency. While these properties are necessary for scalable learning, they are not sufficient for abstraction. Abstraction requires an additional capability: the ability to recognize and stabilize semantic equivalence across distinct construction histories.

We introduce a formal framework in which abstraction is defined operationally rather than behaviorally. The central claim is that abstraction depends on replayable construction: the capacity to reconstruct, inspect, and compare the causal processes by which internal structures were formed. Systems lacking this capacity may generalize statistically, but they cannot reliably abstract.

2 Compression, Representation, and Their Limits

Compression plays a central role in contemporary learning systems. Whether framed in terms of minimum description length, information bottlenecks, or implicit regularization through optimization, successful models reduce high-dimensional inputs to compact internal representations that preserve task-relevant information.

Such representations support interpolation, denoising, and predictive accuracy. However, compression optimizes past performance under a given distribution. It does not, by itself, constrain how representations behave under novel compositions, interventions, or counterfactual variations.

A compressed representation may conflate distinctions that appear irrelevant during training but become critical under new conditions. Conversely, it may preserve distinctions that are statistically salient but semantically accidental. Compression provides no internal criterion for deciding which distinctions should remain stable under future perturbation.

This limitation is not remedied by scale. Increasing model capacity or training data improves approximation within the observed regime but does not address the absence of a principled mechanism for identifying semantic invariants. The issue is not expressivity, but epistemic access to construction.

3 Abstraction as Semantic Equivalence

To distinguish abstraction from compression, we define abstraction in terms of semantic equivalence rather than representational similarity.

Definition 3.1 (Semantic Effect). *A semantic effect is any downstream consequence of an internal structure on the system’s future behavior, inference, or learning dynamics.*

Definition 3.2 (Semantic Equivalence). *Two internal structures are semantically equivalent if, under all admissible continuations of the system’s dynamics, they induce indistinguishable semantic effects.*

Abstraction occurs when a system identifies and commits to such equivalence. This commitment reduces the number of distinct internal representatives required to support future computation, without collapsing distinctions that matter downstream.

Crucially, semantic equivalence is defined counterfactually. It concerns not just what the system has done, but what it would do under alternative trajectories. Determining such equivalence requires access to the causal history by which structures were formed and the ability to reason about alternative histories.

4 Replayable Construction

We now formalize the minimal structural requirement for abstraction.

Definition 4.1 (Construction History). *A construction history is a finite sequence of internal events whose execution yields a given internal structure.*

Definition 4.2 (Replay). *Replay is the capacity of a system to reconstruct the semantic consequences of a construction history, potentially under variation or recombination.*

Replay need not be literal re-execution at the level of implementation. It may be approximate, compressed, or symbolic. What matters is that the system preserves enough causal information to evaluate the consequences of alternative constructions.

Proposition 4.1. *A system that cannot replay construction histories cannot, in general, determine semantic equivalence between distinct internal structures.*

Proof. Semantic equivalence is defined over future consequences under alternative continuations. Without access to construction histories, the system lacks the information needed to evaluate counterfactual continuations. Any equivalence judgment must therefore be heuristic, distribution-dependent, or externally imposed. $\square$

Replay provides a mechanism for grounding abstraction in the system’s own dynamics rather than in external labels or statistical coincidence.

5 Why Compression Alone Is Insufficient

Compression operates on representations, not on construction histories. A compressed representation may encode correlations present in the data, but it does not encode why those correlations arise or whether they will persist under intervention.

This distinction becomes critical when systems encounter distributional shift. Without replayable construction, the system cannot assess whether a previously learned invariant reflects a stable causal structure or an artifact of training data.

From a computational perspective, compression collapses descriptions, while abstraction collapses consequences. The former reduces storage and computation; the latter reduces semantic uncertainty. These are orthogonal objectives.

No amount of compression can substitute for replay, because compression discards information precisely when that information is needed to evaluate counterfactual stability.

6 Relation to Existing Frameworks

Replayable construction is distinct from, though compatible with, several existing approaches in AI.

World models capture predictive structure but often lack explicit access to their own construction history. Causal models encode intervention semantics but typically assume a fixed variable ontology rather than one that is itself constructed and revised. Program induction systems manipulate symbolic structures but rarely preserve the causal trace of their own synthesis.

The present framework differs by treating construction history as a first-class object of computation. Abstraction is not imposed by architectural design or external supervision, but emerges from the system’s ability to compare and stabilize the consequences of alternative constructions.

7 Diagnostic Implications for Artificial Systems

The framework yields a clear diagnostic criterion: a system exhibits abstraction if and only if it can identify semantic equivalence across distinct construction histories.

This criterion explains why systems trained purely on compressed representations often fail under recombination. They lack access to the constructional degrees of freedom that generated those representations.

It also suggests a pathway for improvement. Systems that explicitly preserve and replay construction histories—whether through event logs, program traces, or structured memory—gain the ability to reason about equivalence in a principled way.

Importantly, this does not require symbolic reasoning or human-interpretable representations. Replay operates at the level of internal semantics, not surface form.

8 Discussion

The account presented here reframes abstraction as a structural capability rather than a behavioral outcome. It explains persistent limitations of current systems without appealing to vague notions of understanding or consciousness.

Abstraction is not a matter of having the right representations, but of having the right relation to one’s own construction. Systems that compress without replay may generalize, but they do so blindly. Systems that replay can stabilize meaning across variation.

This distinction clarifies why language-centric systems often inherit abstractions without grounding them, and why biological systems exhibit abstraction prior to language. In both cases, replayable interaction with the environment precedes symbolic expression.

9 Conclusion

We have argued that abstraction requires replayable construction and cannot be reduced to compression or representation learning alone. By formalizing abstraction as semantic equivalence across construction histories, we obtain a precise criterion that distinguishes robust abstraction from statistical generalization.

This framework provides both a theoretical explanation for current limitations in artificial systems and a constructive direction for future architectures. Abstraction is not achieved by scaling compression, but by granting systems access to their own causal past.